NAME: THULASI.S **ROLL:NO:241901118**

EXP 17: CREATE A ROGUE ACCESS POINT TO CAPTURE DATA PACKETS

AIM:

To understand how a Rogue (fake) Access Point / "Evil Twin" attack works and to simulate the setup in a controlled lab to capture and analyse unencrypted test traffic.

THEORY:

A Rogue Access Point is an unauthorised wireless access point that may imitate a trusted network name (SSID). If a user connects to it, traffic routed through that access point may be observed by its operator. Plain HTTP traffic can expose its contents, while HTTPS encrypts application data in transit. The lab manual describes a contained simulation using two lab VMs, hostapd, dnsmasq, and Wireshark.

TOOLS REQUIRED:

- Kali Linux – lab/monitoring machine.
- hostapd – access-point daemon; requires a supported wireless/AP interface.
- dnsmasq – DHCP/DNS service for the isolated test network.
- Wireshark – packet capture and protocol analysis tool.
- A second lab VM acting as the victim/test client, connected only to an isolated lab network.

PROCEDURE:

Step 1 — Install required tools

```
sudo apt update
sudo apt install hostapd dnsmasq -y
```

Step 2 — Configure hostapd (fake AP settings)

Create /etc/hostapd/hostapd.conf:

```
interface=at0
driver=nl80211
ssid=Free_Lab_WiFi
hw_mode=g
channel=6
auth_algs=1
```

```
wpa=0
```

(In this simulated lab, at0 represents a virtual interface bridging the isolated lab network — your instructor will confirm the interface name for your VM setup.)

Step 3 — Configure dnsmasq (to hand out fake IPs to anyone who connects)

Create /etc/dnsmasq.conf:

```
interface=at0
dhcp-range=192.168.50.10,192.168.50.100,12h
dhcp-option=3,192.168.50.1
dhcp-option=6,192.168.50.1
```

Step 4 — Enable IP forwarding and start services

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo systemctl start dnsmasq
sudo hostapd /etc/hostapd/hostapd.conf
```

Step 5 — Connect the "victim" VM to the fake AP

From the victim VM (inside the isolated lab network only), connect to the Free_Lab_WiFi network. It will receive an IP from the attacker's dnsmasq server.

Step 6 — Capture the victim's traffic

On the Kali (attacker) machine:

```
sudo wireshark -i at0
```

Have the victim VM visit an HTTP (not HTTPS) test login page. Watch the packets arrive in Wireshark — filter by:

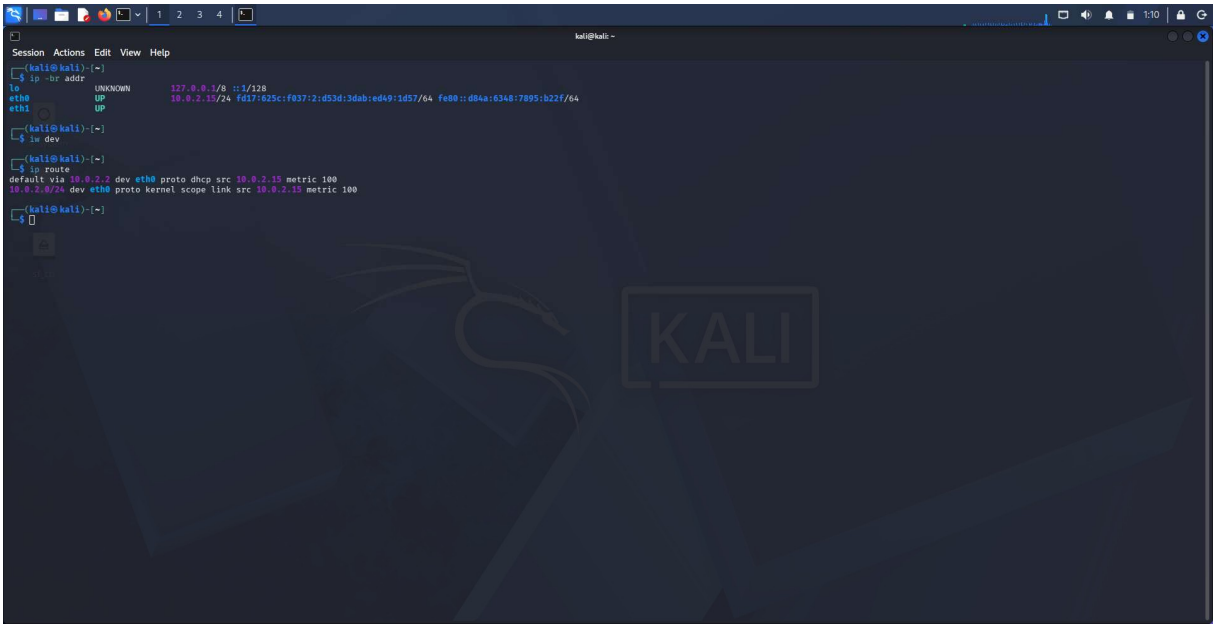
```
http.request.method == "POST"
```

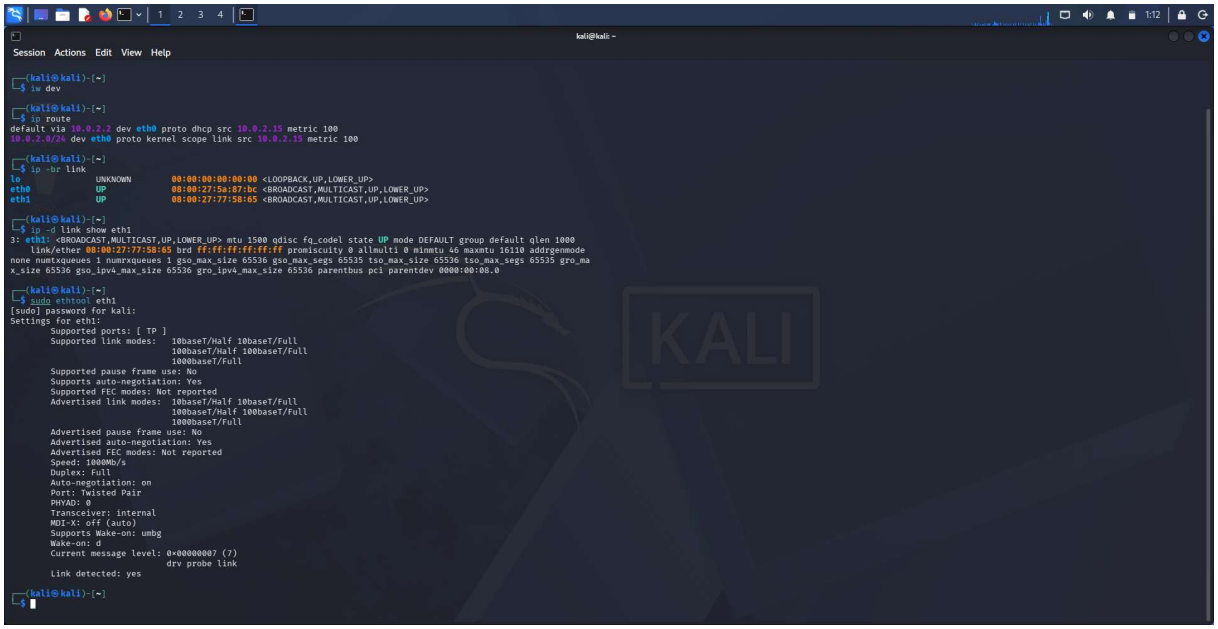
You will see the submitted form data (including credentials, if the test site sends them in plaintext) inside the packet details.

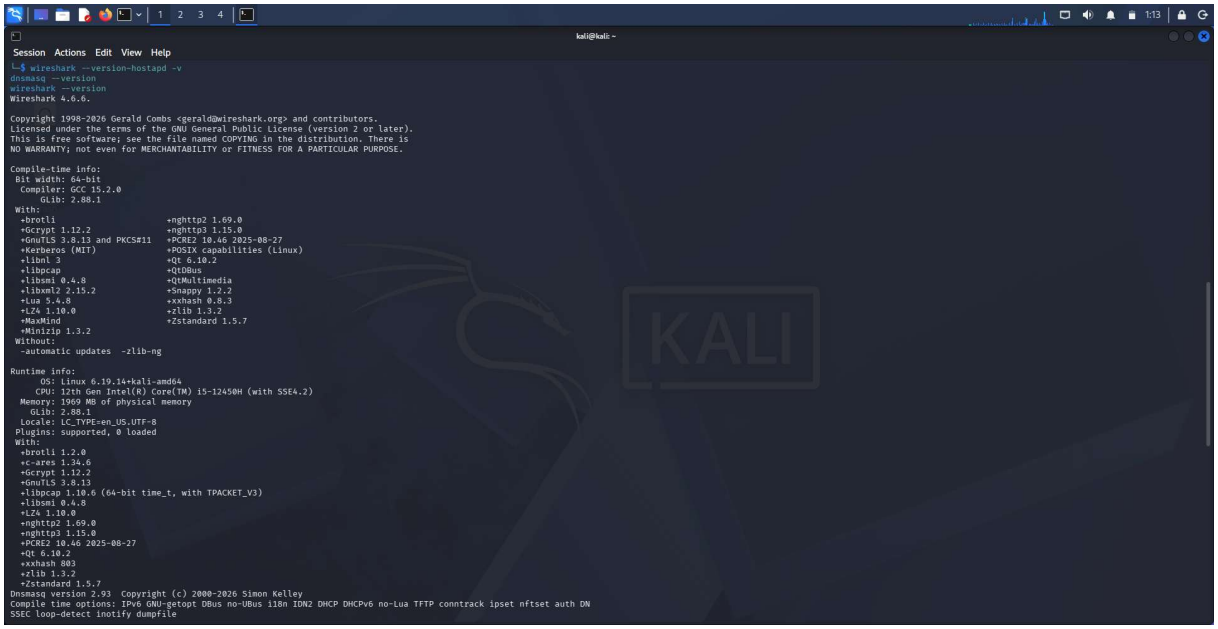
OBSERVATION:

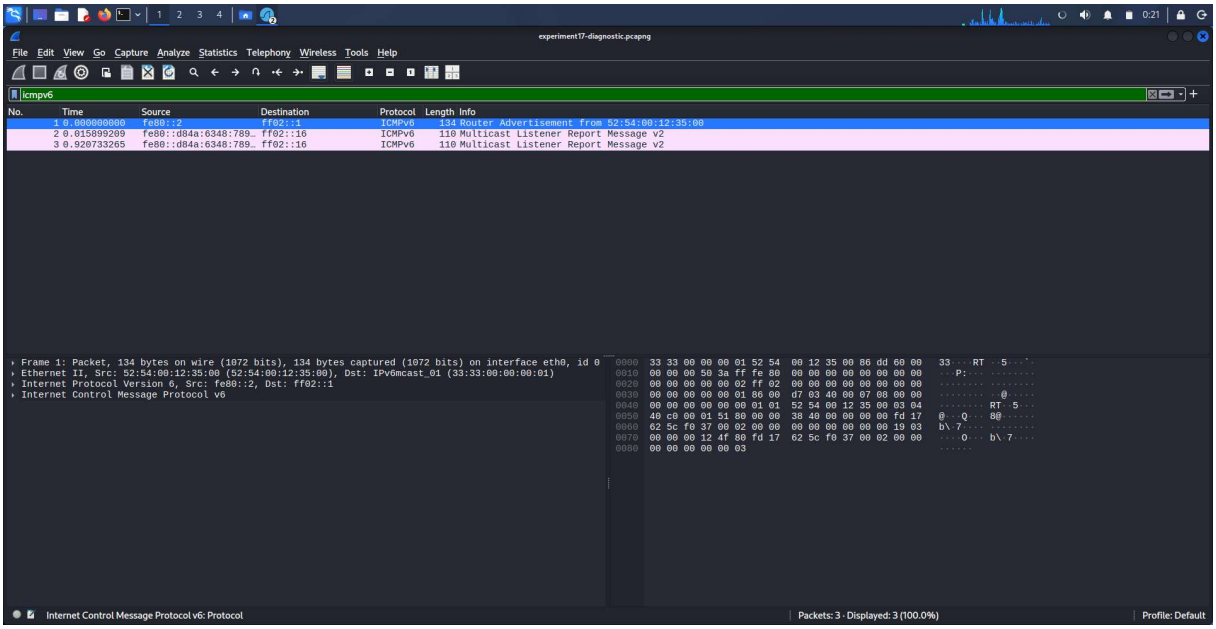
TEST PERFORMED	OBSERVATION
Interface/IP inspection	eth0 is UP with IPv4 address 10.0.2.15/24. eth1 is UP but has no IPv4 address shown in the supplied `ip -br addr` output.
Wireless interface check	`iw dev` returned no output; no wireless interface was detected by this command.
Route inspection	Default route is via 10.0.2.2 on eth0 (VirtualBox NAT network).
Ethernet interface check	`ethtool eth1` reports a detected link at 1000 Mb/s, full duplex; this is a wired Ethernet interface.
Tool-version evidence	The supplied screenshot visibly reports Wireshark 4.6.6 and dnsmasq 2.93. A clear hostapd version result is not visible.
Wireshark capture	The diagnostic capture contains 3 ICMPv6 packets: one Router Advertisement and two Multicast Listener Report messages.
Rogue AP and victim traffic	Not demonstrated in the supplied evidence. No successful hostapd AP startup, victim connection, DHCP lease, HTTP POST, or test-form payload is shown.

OUTPUT:




kali@kali: ~
Session Actions Edit View Help
┌──(kali㉿kali)-[~]
└─$ iw dev
┌──(kali㉿kali)-[~]
└─$ ip route
default via 10.0.2.2 dev eth0 proto dhcp src 10.0.2.15 metric 100
10.0.2.0/24 dev eth0 proto kernel scope link src 10.0.2.15 metric 100
┌──(kali㉿kali)-[~]
└─$ ip -br link
lo UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0 UP 08:00:27:5a:87:bc <BROADCAST,MULTICAST,UP,LOWER_UP>
eth1 UP 08:00:27:77:58:65 <BROADCAST,MULTICAST,UP,LOWER_UP>
┌──(kali㉿kali)-[~]
└─$ ip -d link show eth1
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
link/ether 08:00:27:77:58:65 brd ff:ff:ff:ff:ff:ff promiscuity 0 allmulti 0 minmtu 46 maxmtu 16110 addrgenmode none numtxqueues 1 numrxqueues 1 gso_max_size 65536 gso_max_segs 65535 tso_max_size 65536 tso_max_segs 65535 gro_max_size 65536 gso_ipv4_max_size 65536 gro_ipv4_max_size 65536 parentbus pci parentdev 0000:00:08.0
┌──(kali㉿kali)-[~]
└─$ sudo ethtool eth1
[sudo] password for kali:
Settings for eth1:
Supported ports: [TP]
Supported link modes: 10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
1000baseT/Full
Supported pause frame use: No
Supports auto-negotiation: Yes
Supported FEC modes: Not reported
Advertised link modes: 10baseT/Half 10baseT/Full
100baseT/Half 100baseT/Full
1000baseT/Full
Advertised pause frame use: No
Advertised auto-negotiation: Yes
Advertised FEC modes: Not reported
Speed: 1000Mb/s
Duplex: Full
Auto-negotiation: on
Port: Twisted Pair
PHYAD: 0
Transceiver: internal
MDI-X: off (auto)
Supports Wake-on: umbg
Wake-on: d
Current message level: 0x00000007 (7)
drv probe link
Link detected: yes
┌──(kali㉿kali)-[~]
└─$


kali@kali: ~
Session Actions Edit View Help
└─$ wireshark --version-hostapd -v
dnsmasq --version
wireshark --version
Wireshark 4.6.6.
Copyright 1998-2026 Gerald Combs <gerald@wireshark.org> and contributors.
Licensed under the terms of the GNU General Public License (version 2 or later).
This is free software; see the file named COPYING in the distribution. There is
NO WARRANTY; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
Compile-time info:
Bit width: 64-bit
Compiler: GCC 15.2.0
GLib: 2.88.1
With:
+brotli +nghttp2 1.69.0
+Gcrypt 1.12.2 +nghttp3 1.15.0
+GnuTLS 3.8.13 and PKCS#11 +PCRE2 10.46 2025-08-27
+Kerberos (MIT) +POSIX capabilities (Linux)
+libnl 3 +Qt 6.10.2
+libpcap +QtDBus
+libsmi 0.4.8 +QtMultimedia
+libxml2 2.15.2 +Snappy 1.2.2
+Lua 5.4.8 +xxhash 0.8.3
+LZ4 1.10.0 +zlib 1.3.2
+MaxMind +Zstandard 1.5.7
+Minizip 1.3.2
Without:
--automatic updates -zlib-ng
Runtime info:
OS: Linux 6.19.14+kali-amd64
CPU: 12th Gen Intel(R) Core(TM) i5-12450H (with SSE4.2)
Memory: 1969 MB of physical memory
GLib: 2.88.1
Locale: LC_TYPE=en_US.UTF-8
Plugins: supported, 0 loaded
With:
+brotli 1.2.0
+c-ares 1.34.6
+Gcrypt 1.12.2
+GnuTLS 3.8.13
+libpcap 1.10.6 (64-bit time_t, with TPACKET_V3)
+libsmi 0.4.8
+LZ4 1.10.0
+nghttp2 1.69.0
+nghttp3 1.15.0
+PCRE2 10.46 2025-08-27
+Qt 6.10.2
+xxhash 803
+zlib 1.3.2
+Zstandard 1.5.7
Dnsmasq version 2.93 Copyright (c) 2000-2026 Simon Kelley
Compile time options: IPv6 GNU-getopt DBus no-UBus i18n IDN2 DHCP DHCPv6 no-Lua TFTP conntrack ipset nftset auth DN
SSEC loop-detect inotify dumpfile



RESULT:

The initial network-interface inspection and Wireshark diagnostic capture were completed. The captured evidence confirms that Wireshark displayed three ICMPv6 packets with the icmpv6 filter. The full Rogue Access Point experiment is not yet verified: the current VirtualBox Kali VM exposes Ethernet interfaces only, and no successful AP creation, victim connection, or HTTP test-traffic capture is included. The experiment should be marked complete only after these remaining steps are performed in an isolated, authorised lab.